CampusCommute
Complete Project Blueprint

Student Mobility & Ride-Sharing System

Architecture • Database Schema • Backend • Frontend • DBMS Concepts • Testing • Submission Checklist

PROJECT AT A GLANCE	SPECIFICATION
Application	Campus-focused student ride-sharing and fare sharing
Frontend	HTML, CSS, Bootstrap, JavaScript, Jinja
Backend	Python Flask
Database	MySQL 8.x with InnoDB
Core database	10 relational tables
Main DBMS topics	PK/FK, UNIQUE, CHECK, joins, views, triggers, transactions, normalization

Prepared as a practical build and submission guide. Prioritize a reliable database workflow over optional polish.

1. Project Overview & Architecture

CampusCommute lets students publish rides, discover rides along matching routes, request seats, book rides, share costs and view travel/payment history. The system is designed as a straightforward three-layer web application suitable for a DBMS course project.

Layer	Technology	Responsibilities
Presentation	HTML, CSS, Bootstrap, JavaScript, Jinja	Login/register, dashboard, ride search, publish ride, bookings, payments, reports
Application	Python Flask	Input validation, session/auth flow, route matching, booking/cancellation, reporting
Database	MySQL 8.x / InnoDB	10 tables, relationships, constraints, triggers, views, transactions and analytical queries

Recommended scope

- Do not add microservices, a separate mobile app, live GPS, real-time chat or a real payment gateway for the deadline.
- Use simulated payment records and static/sample locations. Make the core booking workflow functional.
- Use parameterized SQL queries; never concatenate untrusted form values into SQL.

2. Functional Modules

#	Module	Minimum working functionality
1	Student management	Register students; enforce unique email and phone.
2	Driver & vehicle	Register driver details, licence expiry, vehicles and driver–vehicle associations.
3	Locations & routes	Maintain campus/metro/hostel locations and ordered route stops.
4	Ride publishing	Publish a ride with driver, vehicle, departure time and seat capacity.
5	Requests & matching	Search rides where pickup precedes drop on the route and departure time is suitable.
6	Booking & cancellation	Confirm/cancel bookings and update capacity exactly once.
7	Payment & cost sharing	Store fare, payment method, amount, status and timestamp.
8	Reports & analytics	Popular routes, booking counts, driver earnings and seat utilisation.
9	Integrity & transactions	Constraints, triggers, views, transactions and rollback demonstration.

3. Database Schema — 10 Tables

The schema below follows the supplied blueprint, with two practical changes: available_rides gets its own ride_id so one driver can publish multiple rides; bookings gets request_id to preserve request-to-booking traceability. Use InnoDB for foreign keys and transactions.

students

Column	Type	Constraint / meaning
student_id	INT	PK, auto increment
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
phone_number	VARCHAR(15)	UNIQUE, NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
current_mode	VARCHAR(20)	NOT NULL

drivers

Column	Type	Constraint / meaning
student_id	INT	PK and FK → students
licence_id	VARCHAR(50)	UNIQUE, NOT NULL
licence_expiry_date	DATE	NOT NULL
upi_id	VARCHAR(100)	Nullable

vehicle

Column	Type	Constraint / meaning
vehicle_number	VARCHAR(20)	PK
type	VARCHAR(30)	NOT NULL
name	VARCHAR(60)	NOT NULL
colour	VARCHAR(30)	Nullable
year_of_manufacture	YEAR	Nullable
capacity	INT	CHECK > 0

driver_vehicle

Column	Type	Constraint / meaning
student_id	INT	Composite PK; FK → drivers
vehicle_number	VARCHAR(20)	Composite PK; FK → vehicle

locations

Column	Type	Constraint / meaning
location_id	INT	PK, auto increment
name	VARCHAR(100)	NOT NULL
latitude	DECIMAL(10,7)	Nullable
longitude	DECIMAL(10,7)	Nullable

available_rides

Column	Type	Constraint / meaning
ride_id	INT	PK, auto increment
driver_id	INT	FK → drivers
vehicle_number	VARCHAR(20)	FK → vehicle
departure_time	DATETIME	NOT NULL
total_capacity	INT	NOT NULL
remaining_capacity	INT	NOT NULL
status	VARCHAR(20)	NOT NULL

ride_points

Column	Type	Constraint / meaning
ride_id	INT	Composite PK; FK → available_rides
sequence_no	INT	Composite PK
location_id	INT	FK → locations

requests

Column	Type	Constraint / meaning
request_id	INT	PK, auto increment
customer_id	INT	FK → students
pickup_location_id	INT	FK → locations
drop_location_id	INT	FK → locations
schedule_time	DATETIME	NOT NULL
status	VARCHAR(20)	NOT NULL

bookings

Column	Type	Constraint / meaning
booking_id	INT	PK, auto increment
request_id	INT	FK → requests, UNIQUE
customer_id	INT	FK → students
ride_id	INT	FK → available_rides
pickup_location_id	INT	FK → locations
drop_location_id	INT	FK → locations
booking_time	DATETIME	NOT NULL
status	VARCHAR(20)	NOT NULL
fare	DECIMAL(10,2)	NOT NULL

payment

Column	Type	Constraint / meaning
payment_id	INT	PK, auto increment
booking_id	INT	FK → bookings
payment_date	DATETIME	Nullable until recorded
amount	DECIMAL(10,2)	NOT NULL
status	VARCHAR(20)	NOT NULL
method	VARCHAR(20)	NOT NULL

Integrity rules

- remaining_capacity must be between zero and total_capacity; fare and payment amount must be non-negative.
- Use consistent status values appropriate to each table, such as PENDING, CONFIRMED, CANCELLED, PUBLISHED, PAID and FAILED.
- A unique request_id on bookings means one request can create at most one booking. Payment can be one-to-many if payment attempts must be retained.

4. ER Relationships & Data Flow

Relationship	Cardinality / implementation
Students → Drivers	1 to 0..1; drivers.student_id is both PK and FK.
Drivers ↔ Vehicles	Many-to-many through driver_vehicle.
Drivers → Available rides	One driver can publish many rides.
Available rides → Ride points	One ride has many ordered stops.
Locations → Ride points	A location can appear in many ride routes.
Students → Requests	A student can create many requests.
Requests → Bookings	A request can have zero or one booking (UNIQUE request_id).
Available rides → Bookings	One ride can have many bookings.
Bookings → Payment	One booking can have one or many payment records.
Locations → Requests / Bookings	Each request/booking references pickup and drop locations.

Passenger flow

Student → Create request → Match ordered ride stops → Create booking → Trigger adjusts seat capacity → Record payment → Display booking history and reports.

5. Backend Routes / Endpoints

Feature	Endpoint	Method
Student registration	/register	POST
Login	/login	POST
Driver registration	/driver/register	POST
Add vehicle	/vehicles/add	POST
Publish ride	/rides/publish	POST
Search rides	/rides/search	GET
Create request	/requests/create	POST
Create booking	/bookings/create	POST
Cancel booking	/bookings/cancel/	POST
Record payment	/payments/create	POST
Reports	/reports	GET

Business logic rules

- Ride matching: pickup stop sequence must be less than drop stop sequence; departure must fit the requested time.
- Booking: validate seat availability and insert the booking inside one transaction. Capacity must not be decremented in both Flask and a trigger.
- Cancellation: restore seats only when a booking transitions from CONFIRMED to CANCELLED, never twice.
- Licence validation: block publishing when licence_expiry_date is earlier than the current date.
- Payment: store a simulated payment record; do not imply real money moved.

6. Frontend Pages

Page	Fields / contents
Login / Register	Student name, email, phone, login/register forms.
Dashboard	Ride search, publish ride, upcoming rides and bookings.
Find a ride	Pickup, drop, departure time, results, available seats and fare.
Publish a ride	Vehicle, departure time, ordered stops, capacity and fare setup.
My bookings	Booking ID, driver, route, fare, booking status, cancel action, payment status.
Reports	Popular routes, bookings per day, driver earnings and seat utilisation.

Suggested design

- Use a consistent navigation bar, cards for rides, tables for booking history and clear status labels.
- Use Bootstrap to save time. Charts are optional; SQL output tables are enough for DBMS evaluation.

7. DBMS Concepts to Demonstrate

Constraints

- PRIMARY KEY uniquely identifies a row; FOREIGN KEY preserves referential integrity; UNIQUE prevents duplicate email, phone and licence IDs.
- NOT NULL marks required fields; CHECK enforces valid capacity and non-negative fare; indexes help common searches.

Example SQL queries

```
-- 1. Rides with driver and vehicle
SELECT ar.ride_id, s.first_name, v.name,
       ar.departure_time, ar.remaining_capacity
FROM available_rides ar
JOIN drivers d ON d.student_id = ar.driver_id
JOIN students s ON s.student_id = d.student_id
JOIN vehicle v ON v.vehicle_number = ar.vehicle_number
WHERE ar.status = 'PUBLISHED'
  AND ar.remaining_capacity > 0;
```

```
-- 2. Count confirmed bookings per ride
SELECT ride_id, COUNT(*) AS total_bookings
FROM bookings
WHERE status = 'CONFIRMED'
GROUP BY ride_id;
```

```
-- 3. Popular pickup locations
SELECT l.name, COUNT(*) AS total_pickups
FROM bookings b
JOIN locations l ON l.location_id = b.pickup_location_id
WHERE b.status = 'CONFIRMED'
GROUP BY l.location_id, l.name
ORDER BY total_pickups DESC;
```

```
-- 4. Confirmed booking fares by driver (not necessarily paid revenue)
SELECT ar.driver_id, SUM(b.fare) AS total_fares
FROM available_rides ar
JOIN bookings b ON b.ride_id = ar.ride_id
WHERE b.status = 'CONFIRMED'
GROUP BY ar.driver_id;
```

```
-- 5. Payment history
SELECT b.booking_id, b.fare, p.method,
       p.status AS payment_status
FROM bookings b
LEFT JOIN payment p ON p.booking_id = b.booking_id;
```

```
-- 6. Rides without confirmed bookings
SELECT ar.ride_id
FROM available_rides ar
LEFT JOIN bookings b
  ON b.ride_id = ar.ride_id
```

```
 AND b.status = 'CONFIRMED'
WHERE b.booking_id IS NULL;
```

Views

```
CREATE VIEW confirmed_bookings_view AS
SELECT booking_id, customer_id, ride_id,
       pickup_location_id, drop_location_id, fare
FROM bookings
WHERE status = 'CONFIRMED';

CREATE VIEW published_rides_view AS
SELECT ride_id, driver_id, vehicle_number,
       departure_time, remaining_capacity
FROM available_rides
WHERE status = 'PUBLISHED'
  AND remaining_capacity > 0;
```

Triggers

- Capacity trigger: enforce capacity and update remaining_capacity when a booking becomes CONFIRMED; restore capacity on cancellation.
- Licence trigger: reject a new ride if the driver's licence has expired.

Transactions

```
START TRANSACTION;
-- Insert booking; trigger updates capacity.
-- If any step fails, application issues ROLLBACK.
COMMIT;
```

Normalization

- 1NF: attributes are atomic; route stops are separate rows.
- 2NF: non-key attributes depend on the whole composite key.
- 3NF: student, driver, vehicle and location details are stored in their own relations, avoiding unnecessary duplication.
- BCNF requires every determinant in a relation to be a candidate key; state this only if functional dependencies confirm it.

8. Recommended Folder Structure

```
CampusCommute/
■■■ app.py
■■■ config.py
■■■ requirements.txt
■■■ README.md
■■■ database/
■   ■■■ schema.sql
■   ■■■ seed.sql
■   ■■■ views.sql
■   ■■■ triggers.sql
■   ■■■ reports.sql
■■■ routes/
■   ■■■ auth.py
■   ■■■ students.py
■   ■■■ drivers.py
■   ■■■ rides.py
■   ■■■ bookings.py
■   ■■■ reports.py
■■■ templates/
■   ■■■ base.html
■   ■■■ login.html
■   ■■■ register.html
■   ■■■ dashboard.html
■   ■■■ rides.html
■   ■■■ publish_ride.html
■   ■■■ bookings.html
■   ■■■ reports.html
■■■ static/
    ■■■ css/style.css
    ■■■ js/app.js
```

9. Setup & Build Sequence

Step	Work to finish
1. Database	Create schema, constraints, seed records; verify foreign keys.
2. Backend	Connect Flask to MySQL; test registration and reading data.
3. Core workflow	Publish rides, match routes, book and cancel; confirm capacity changes.
4. DBMS features	Implement views, triggers, reports, transactions and rollback tests.
5. Frontend & submission	Finish templates, screenshots, README, report and PPT.

Install and run

```
python -m venv .venv
source .venv/bin/activate       # macOS/Linux
pip install Flask mysql-connector-python
pip freeze > requirements.txt
python app.py
```

Windows activation: .venv\\Scripts\\activate. Configure credentials through environment variables or a local config file that is not submitted publicly.

10. Testing Checklist

Test	Expected result
Duplicate email	Rejected by UNIQUE constraint.
Valid driver licence	Ride can be published.
Expired licence	Ride publishing rejected.
Pickup before drop	Ride qualifies for matching.
Pickup after drop	Ride does not qualify.
Booking with seats	Confirmed; capacity decreases once.
Booking without seats	Rejected; capacity unchanged.
Cancel confirmed booking	Capacity restored once.
Cancel same booking again	No duplicate restoration.
Payment record	Visible in booking/payment history.
Analytics	Aggregates return expected results.
Rollback	Uncommitted changes are undone.

11. Submission Deliverables

- Working Flask source code and all required templates/static files.
- schema.sql with all 10 tables, keys and constraints.
- seed.sql with enough realistic records for a complete demonstration.
- views.sql, triggers.sql and reports.sql.
- ER diagram and relational schema diagram.
- Project report: problem, objectives, related systems, proposed solution, architecture, schema, normalization, implementation and results.
- PPT with the ER diagram and screenshots of the working system.
- README with setup, database creation and execution instructions.
- Screenshots showing UI, SQL queries, trigger behavior, views and reports.

12. Demo Sequence

Order	Demonstration
1	Register a student and show the student record.
2	Register driver details and a vehicle.
3	Publish a ride with ordered stops.
4	Search a valid pickup-to-drop route.
5	Book a seat and show capacity decrement.
6	Cancel the booking and show capacity restoration.
7	Record a simulated payment.
8	Run a JOIN and aggregate report.
9	Show a view, trigger and transaction/rollback.

13. Deadline Priorities

Priority	What must be ready
Must work	Database, sample data, ride publishing/search, booking and capacity management.
Must demonstrate	Joins, aggregates, views, triggers, transactions and constraints.
Must prepare	ER diagram, report, PPT, README and screenshots.
Optional	Charts, advanced styling, maps, notifications and extra admin features.

Final recommendation

Build database-first, then backend, then frontend. Every feature shown in the demo should correspond to a real database operation. A small, reliable end-to-end workflow is stronger than a large interface with non-working buttons.